In [1]:
```
import pandas as pd

df = pd.read_csv('/content/auto-mpg[1].csv')

display(df.head())

display(df.info())

display(df.isnull())
```

	mpg	cylinders	displacement	horsepower	weight	acceleration	model year	origi
0	18.0	8	307.0	130	3504	12.0	70	1
1	15.0	8	350.0	165	3693	11.5	70	1
2	18.0	8	318.0	150	3436	11.0	70	1
3	16.0	8	304.0	150	3433	12.0	70	1
4	17.0	8	302.0	140	3449	10.5	70	1

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 398 entries, 0 to 397
Data columns (total 9 columns):
 #   Column        Non-Null Count  Dtype  
---  ------        --------------  -----  
 0   mpg           398 non-null    float64
 1   cylinders     398 non-null    int64  
 2   displacement  398 non-null    float64
 3   horsepower    398 non-null    object 
 4   weight        398 non-null    int64  
 5   acceleration  398 non-null    float64
 6   model year    398 non-null    int64  
 7   origin        398 non-null    int64  
 8   car name      398 non-null    object 
dtypes: float64(3), int64(4), object(2)
memory usage: 28.1+ KB
None
```

	mpg	cylinders	displacement	horsepower	weight	acceleration	model year	ori
0	False	False	False	False	False	False	False	Fa
1	False	False	False	False	False	False	False	Fa
2	False	False	False	False	False	False	False	Fa
3	False	False	False	False	False	False	False	Fa
4	False	False	False	False	False	False	False	Fa
...	...	...	...	...	...	...	...	
393	False	False	False	False	False	False	False	Fa
394	False	False	False	False	False	False	False	Fa
395	False	False	False	False	False	False	False	Fa
396	False	False	False	False	False	False	False	Fa
397	False	False	False	False	False	False	False	Fa

398 rows × 9 columns

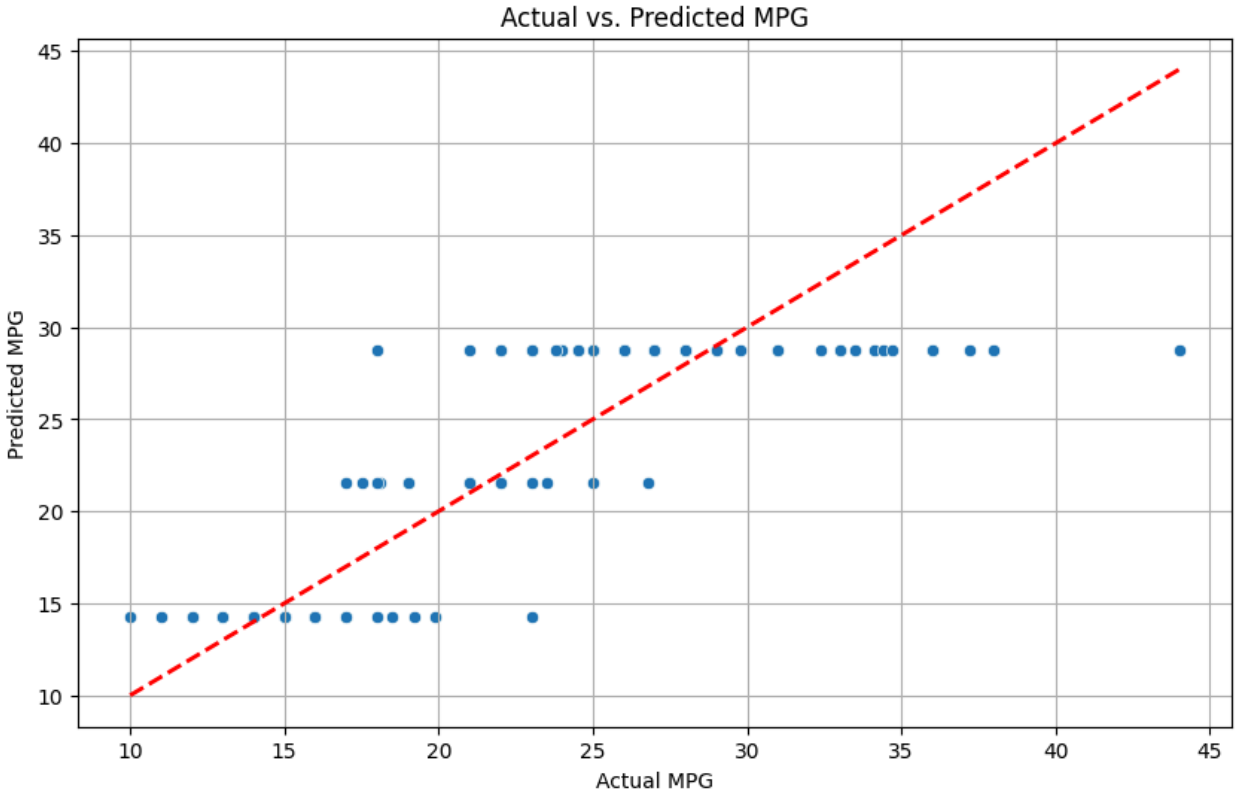
In [12]:
```
import matplotlib.pyplot as plt
import seaborn as sns

df['horsepower'] = pd.to_numeric(df['horsepower'], errors='coerce')
df['horsepower'] = df['horsepower'].fillna(df['horsepower'].median())

display(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 398 entries, 0 to 397
Data columns (total 9 columns):
 #   Column        Non-Null Count  Dtype  
---  ------        --------------  -----  
 0   mpg           398 non-null    float64
 1   cylinders     398 non-null    int64  
 2   displacement  398 non-null    float64
 3   horsepower    398 non-null    float64
 4   weight        398 non-null    int64  
 5   acceleration  398 non-null    float64
 6   model year    398 non-null    int64  
 7   origin        398 non-null    int64  
 8   car name      398 non-null    object 
dtypes: float64(4), int64(4), object(1)
memory usage: 28.1+ KB
None
```

In [14]:
```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
```

```
X = df[['cylinders']]
y = df['mpg']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, randc

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Mean Absolute Error (MAE):", mean_absolute_error(y_test, y_pred))
print("Mean Squared Error (MSE):", mean_squared_error(y_test, y_pred))
print("R-squared (R2):", r2_score(y_test, y_pred))

plt.figure(figsize=(10, 6))
sns.scatterplot(x=y_test, y=y_pred)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw
plt.xlabel('Actual MPG')
plt.ylabel('Predicted MPG')
plt.title('Actual vs. Predicted MPG')
plt.grid(True)
plt.show()
```

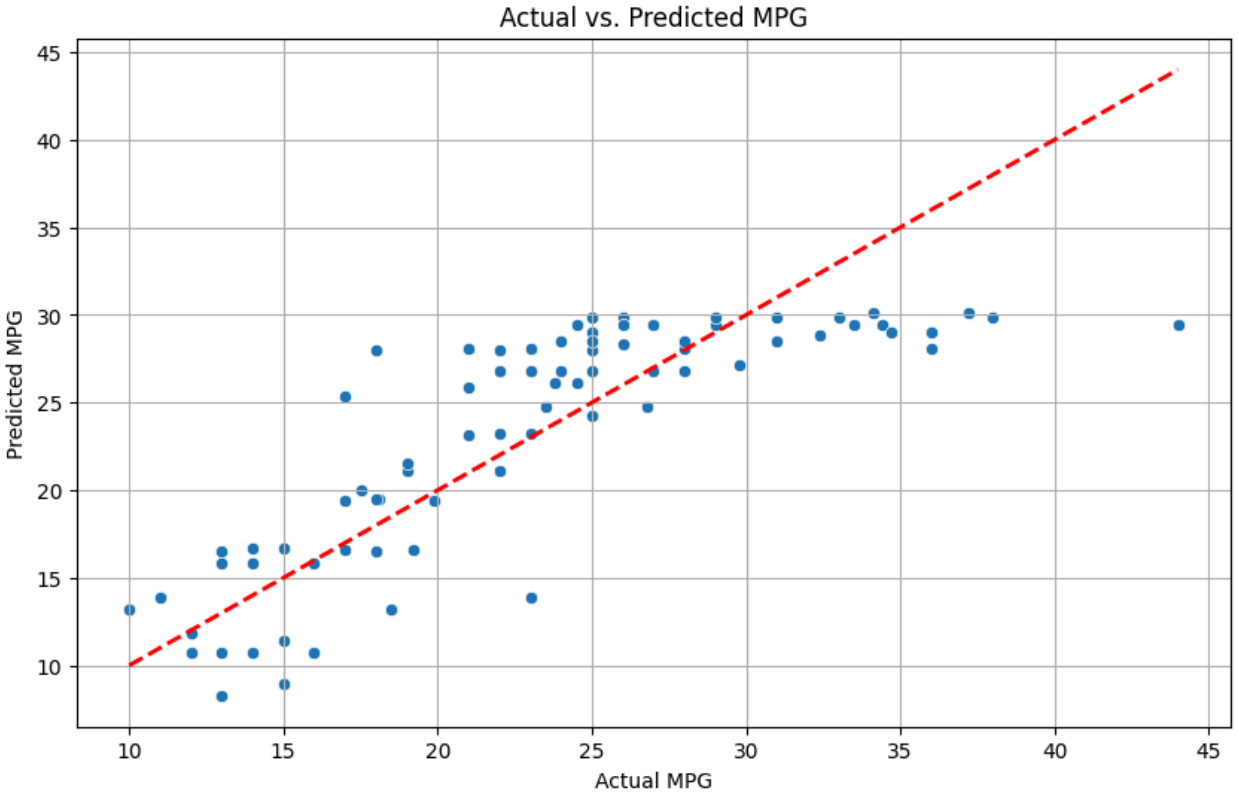
```
Mean Absolute Error (MAE): 3.5184964254890927
Mean Squared Error (MSE): 19.6648641570341
R-squared (R2): 0.6342539549673285
```



In [15]:
```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
```

```
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error

X = df[['displacement']]
y = df['mpg']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, randc

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Mean Absolute Error (MAE):", mean_absolute_error(y_test, y_pred))
print("Mean Squared Error (MSE):", mean_squared_error(y_test, y_pred))
print("R-squared (R2):", r2_score(y_test, y_pred))

plt.figure(figsize=(10, 6))
sns.scatterplot(x=y_test, y=y_pred)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw
plt.xlabel('Actual MPG')
plt.ylabel('Predicted MPG')
plt.title('Actual vs. Predicted MPG')
plt.grid(True)
plt.show()
```

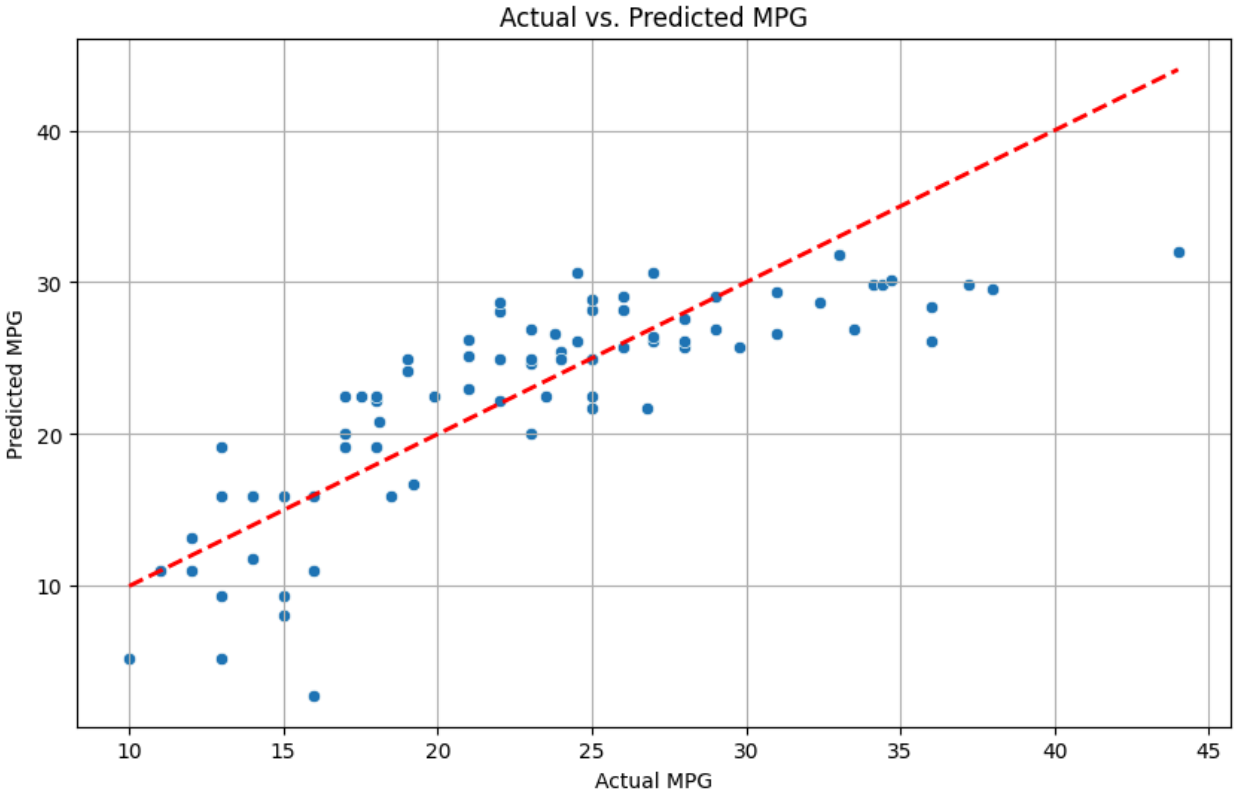
```
Mean Absolute Error (MAE): 3.3949595383649607
Mean Squared Error (MSE): 18.102543998358946
R-squared (R2): 0.6633114869465596
```



In [16]:
```
from sklearn.model_selection import train_test_split
```

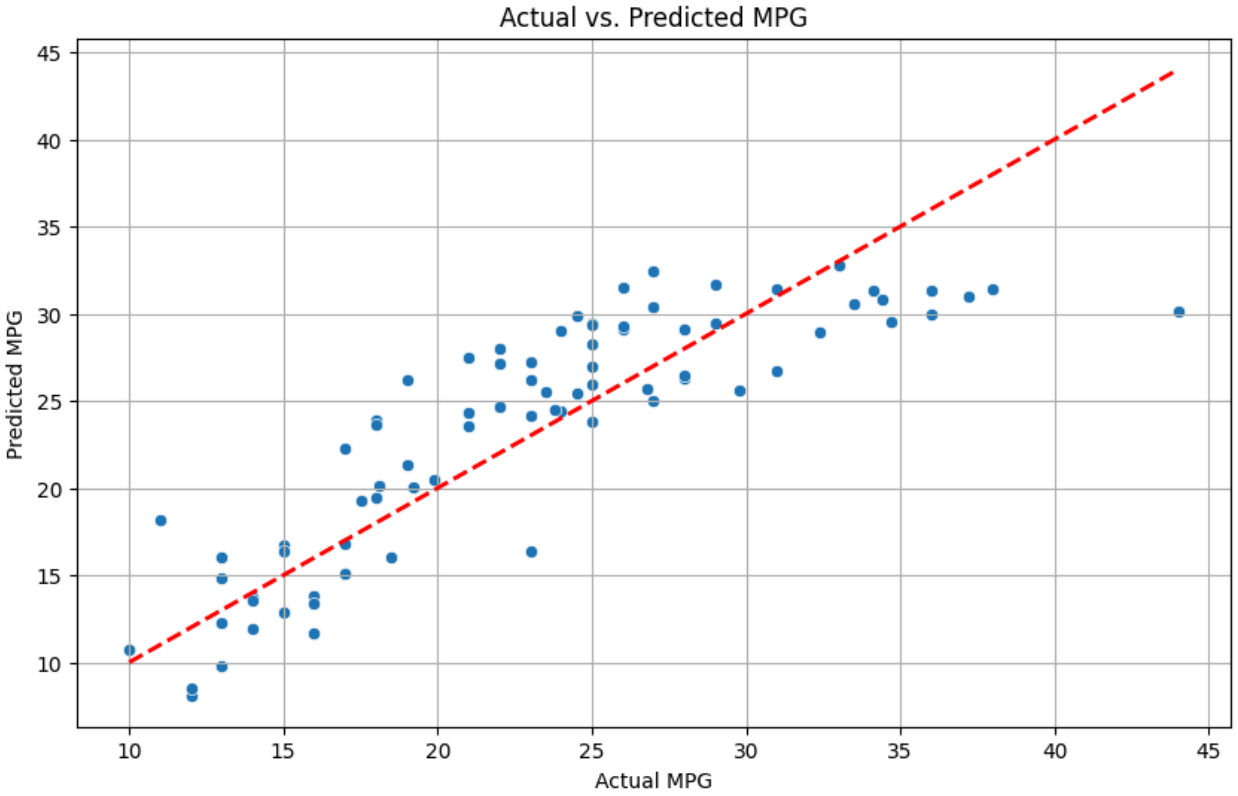
```python
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error

X = df[['horsepower']]
y = df['mpg']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, randc

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Mean Absolute Error (MAE):", mean_absolute_error(y_test, y_pred))
print("Mean Squared Error (MSE):", mean_squared_error(y_test, y_pred))
print("R-squared (R2):", r2_score(y_test, y_pred))

plt.figure(figsize=(10, 6))
sns.scatterplot(x=y_test, y=y_pred)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw
plt.xlabel('Actual MPG')
plt.ylabel('Predicted MPG')
plt.title('Actual vs. Predicted MPG')
plt.grid(True)
plt.show()
```

```
Mean Absolute Error (MAE): 3.5089056305441026
Mean Squared Error (MSE): 19.37313898265924
R-squared (R2): 0.6396797401602512
```



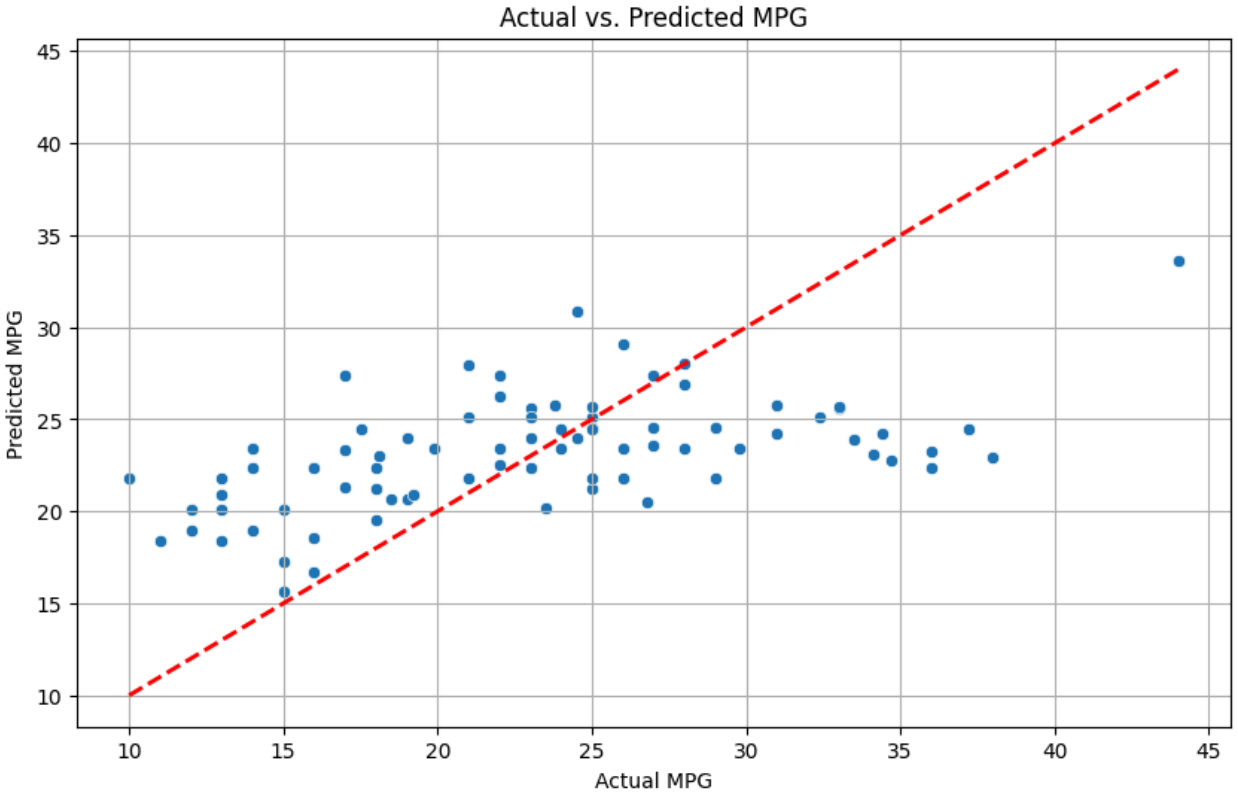
In [17]:
```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error

X = df[['weight']]
y = df['mpg']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, randc

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Mean Absolute Error (MAE):", mean_absolute_error(y_test, y_pred))
print("Mean Squared Error (MSE):", mean_squared_error(y_test, y_pred))
print("R-squared (R2):", r2_score(y_test, y_pred))

plt.figure(figsize=(10, 6))
sns.scatterplot(x=y_test, y=y_pred)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw
plt.xlabel('Actual MPG')
plt.ylabel('Predicted MPG')
plt.title('Actual vs. Predicted MPG')
plt.grid(True)
plt.show()
```

```
Mean Absolute Error (MAE): 3.1177861992064573
Mean Squared Error (MSE): 14.894861064636194
R-squared (R2): 0.722971057303075
```



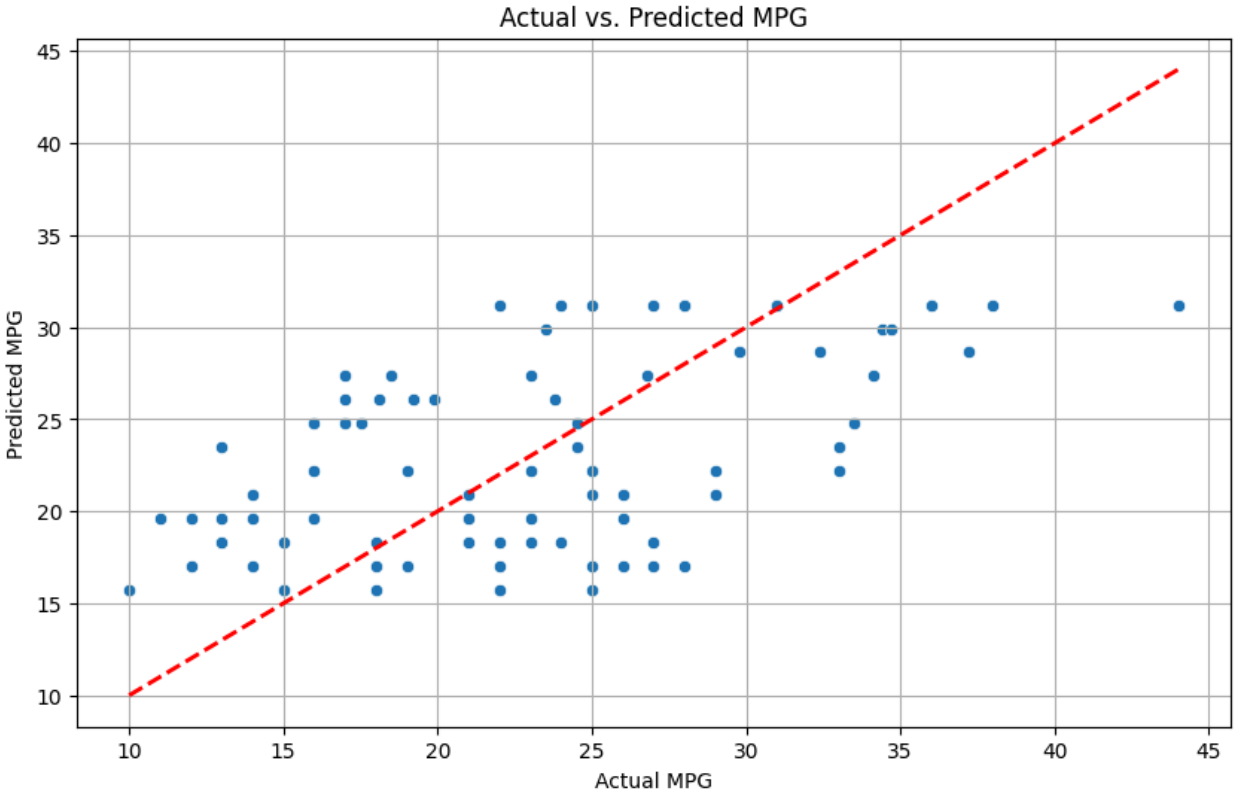
In [18]:
```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error

X = df[['acceleration']]
y = df['mpg']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, randc

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Mean Absolute Error (MAE):", mean_absolute_error(y_test, y_pred))
print("Mean Squared Error (MSE):", mean_squared_error(y_test, y_pred))
print("R-squared (R2):", r2_score(y_test, y_pred))

plt.figure(figsize=(10, 6))
sns.scatterplot(x=y_test, y=y_pred)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw
plt.xlabel('Actual MPG')
plt.ylabel('Predicted MPG')
plt.title('Actual vs. Predicted MPG')
plt.grid(True)
plt.show()
```

```
Mean Absolute Error (MAE): 4.985506778771171
Mean Squared Error (MSE): 38.509179200895026
R-squared (R2): 0.2837692710354305
```



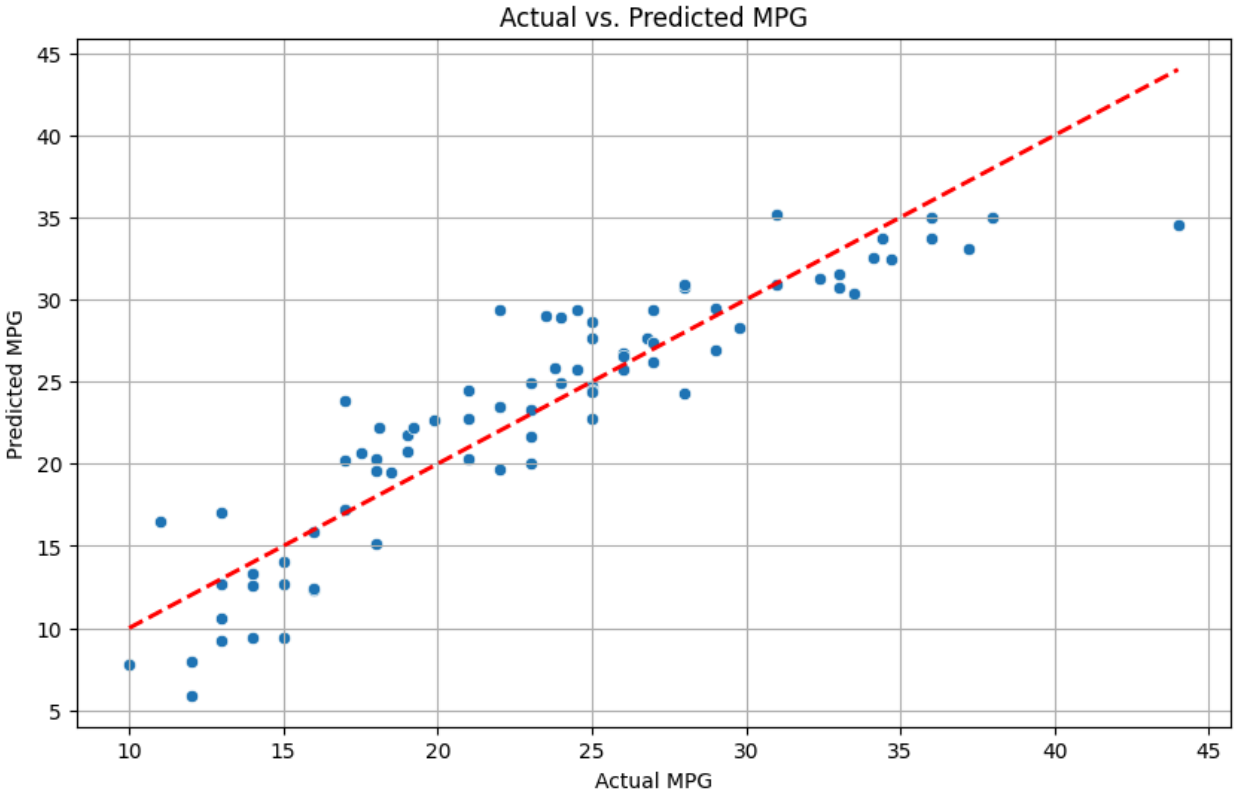
In [19]:
```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error

X = df[['model year']]
y = df['mpg']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, randc

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Mean Absolute Error (MAE):", mean_absolute_error(y_test, y_pred))
print("Mean Squared Error (MSE):", mean_squared_error(y_test, y_pred))
print("R-squared (R2):", r2_score(y_test, y_pred))

plt.figure(figsize=(10, 6))
sns.scatterplot(x=y_test, y=y_pred)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw
plt.xlabel('Actual MPG')
plt.ylabel('Predicted MPG')
plt.title('Actual vs. Predicted MPG')
plt.grid(True)
plt.show()
```

```
Mean Absolute Error (MAE): 5.357206585199805
Mean Squared Error (MSE): 38.320749420819155
R-squared (R2): 0.28727386919989206
```



In [20]:
```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error

X = df[['cylinders', 'displacement', 'horsepower', 'weight', 'acceleration', '
y = df['mpg']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, randc

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Mean Absolute Error (MAE):", mean_absolute_error(y_test, y_pred))
print("Mean Squared Error (MSE):", mean_squared_error(y_test, y_pred))
print("R-squared (R2):", r2_score(y_test, y_pred))

plt.figure(figsize=(10, 6))
sns.scatterplot(x=y_test, y=y_pred)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw
plt.xlabel('Actual MPG')
plt.ylabel('Predicted MPG')
plt.title('Actual vs. Predicted MPG')
plt.grid(True)
plt.show()
```

```
Mean Absolute Error (MAE): 2.4667808004520344
Mean Squared Error (MSE): 9.440068465263394
R-squared (R2): 0.8244245330943356
```



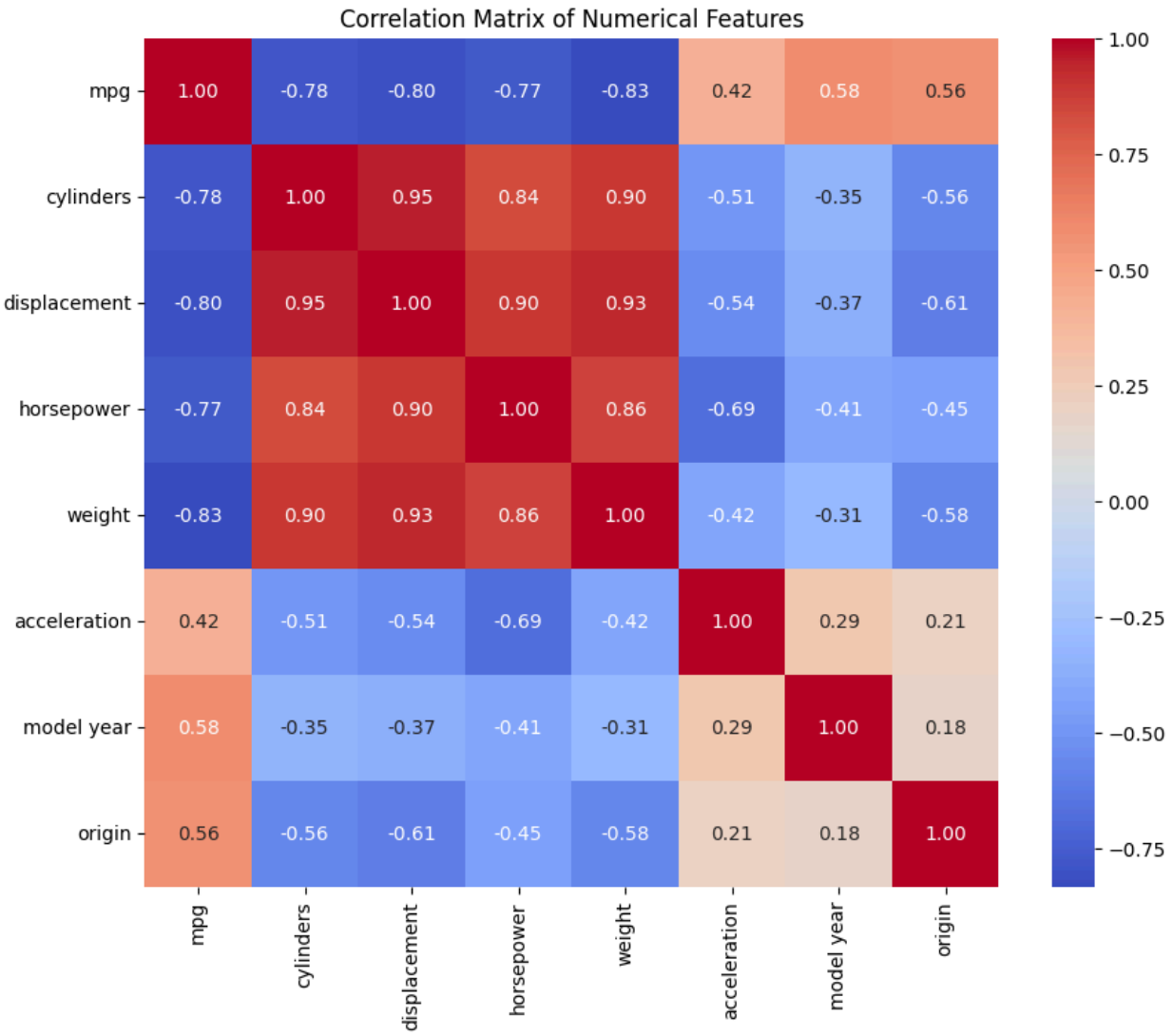
In [22]:
```
predictions_df = pd.DataFrame({'Actual MPG': y_test, 'Predicted MPG': y_pred})
predictions_df = predictions_df.reset_index(drop=True)
display(predictions_df.head(20))
```

	Actual MPG	Predicted MPG
0	33.0	31.530425
1	28.0	30.710936
2	19.0	21.759651
3	13.0	17.004440
4	14.0	12.613165
5	27.0	26.215144
6	24.0	28.912845
7	13.0	9.207272
8	17.0	17.219187
9	21.0	22.718517
10	15.0	12.682684
11	38.0	34.988286
12	26.0	26.695783
13	15.0	14.058199
14	25.0	24.776141
15	12.0	5.902393
16	31.0	30.874208
17	17.0	23.815270
18	16.0	15.882718
19	31.0	35.165563

Correlation Matrix of Numerical Features

In [23]:
```
import matplotlib.pyplot as plt
import seaborn as sns

# Select numerical columns for correlation matrix
numerical_cols = ['mpg', 'cylinders', 'displacement', 'horsepower', 'weight',
correlation_matrix = df[numerical_cols].corr()

plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Correlation Matrix of Numerical Features')
```

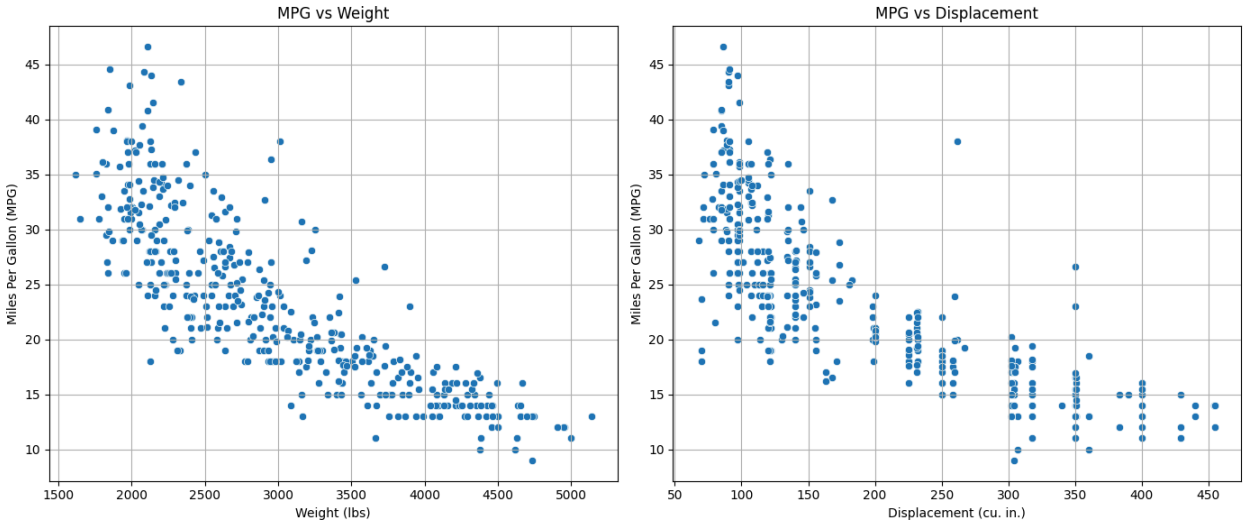
```
plt.show()
```



Relationship between MPG and Key Predictors

In [24]:
```
plt.figure(figsize=(14, 6))

# MPG vs Weight
plt.subplot(1, 2, 1) # 1 row, 2 columns, 1st plot
sns.scatterplot(x='weight', y='mpg', data=df)
plt.title('MPG vs Weight')
plt.xlabel('Weight (lbs)')
plt.ylabel('Miles Per Gallon (MPG)')
plt.grid(True)

# MPG vs Displacement
plt.subplot(1, 2, 2) # 1 row, 2 columns, 2nd plot
sns.scatterplot(x='displacement', y='mpg', data=df)
plt.title('MPG vs Displacement')
plt.xlabel('Displacement (cu. in.)')
```

```
plt.ylabel('Miles Per Gallon (MPG)')
plt.grid(True)

plt.tight_layout()
plt.show()
```



Key Insights from Linear Regression Analysis

1. **Most Influential Single Predictor**: The `weight` feature was found to be the strongest individual predictor of `mpg`, achieving an R-squared value of approximately 0.72. This suggests that vehicle weight has a significant inverse relationship with fuel efficiency.
2. **Moderate Predictors**: `displacement`, `horsepower`, and `cylinders` also showed moderate predictive power, with R-squared values ranging between 0.63 and 0.66. These engine-related characteristics are important factors in determining `mpg`.
3. **Weak Predictors**: `acceleration` and `model year` were the weakest individual predictors, with R-squared values around 0.28. While they do have some correlation, their individual contribution to explaining `mpg` variance is limited.
4. **Combined Feature Performance**: The linear regression model utilizing all six numerical features (`cylinders`, `displacement`, `horsepower`, `weight`, `acceleration`, `model year`) significantly improved the prediction accuracy, yielding an R-squared value of approximately 0.82. This indicates that combining these features provides a much more comprehensive understanding of the factors influencing `mpg` compared to any single feature alone.
5. **Model Effectiveness**: The overall model demonstrates good predictive capability, explaining about 82% of the variance in `mpg` based on the selected features.